Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 7

Morfološke operacije. *Watershed* algoritam.

Tema ovog tutorijala su morfološke operacije i *Watershed* algoritam. Pokazati ćemo kako se može u OpenCV-u implementirati erozija, dilatacija, napredne morfološke operacije i *Watershed* algoritam.

Radi olakšanja načina programiranja, napravit ćemo funkcije koje će biti implementirane da bi se olakšalo shvatanje i implementacija koda. Ovaj kod je zasnovan na kodu sa OpenCV stranice koji je prikazan na linku.

Prvi dio `main` funkcije je jednostavan - potrebno je učitati sliku nad kojom želimo izvršiti *Watershed* algoritam, prebaciti je u *grayscaele* format, i izvršiti *Otsu*-ovu binarizaciju sa funkcijom `cv::threshold` gdje će posljednji parametar biti `cv::THRESH_BINARY_INV + cv::THRESH_OTSU`.

Nakon toga je potrebno kreirati jedan prozor sa tri *trackbar*-a (ovo je da bude lakše pratiti šta se dešava sa slikom) na sljedeći način:

```
1     cv::namedWindow(window_name, cv::WINDOW_AUTOSIZE);
2     cv::createTrackbar("Operator:\n 0: Opening - 1: Closing  \n 2: Gradient
          - 3: Top Hat \n 4: Black Hat", window_name, &morph_operator,
          max_operator, Morfoloske_Operacije);
3     cv::createTrackbar("Element:\n 0: Rect - 1: Cross - 2: Ellipse",
          window_name, &morph_elem, max_elem, Morfoloske_Operacije);
4     cv::createTrackbar("Kernel size:\n 2n +1", window_name, &morph_size,
          max_kernel_size, Morfoloske_Operacije);
```

Bitno je naglasiti da sve korištene varijable (uključujući i učitane slike) su globalne varijable! One su navedene u nastavku, iznad `main` funkcije, zajedno sa prototipima prethodno navedenih funkcija:

```
1  cv::Mat originalna, gray, binarna, morfoloska_obrada, dilatacija, erozija,
       razlika, watershed;
2
3  int morph_operator = 0;
4  int morph_elem = 0;
5  int morph_size = 0;
6
7  int erosion_elem = 0;
8  int erosion_size = 0;
9  int dilation_elem = 0;
10 int dilation_size = 0;
11
12 int const max_operator = 4;
13 int const max_elem = 2;
14 int const max_kernel_size = 21;
15
16 std::string window_name = "Morfoloska transformacija";
17
18 void Morfoloske_Operacije(int, void*);
19 void Dilatacija(int, void*);
20 void Erozija(int, void*);
```

Vratimo se na *trackbar*-ove koje smo prethodno napravili. Kao što možete primijetiti, oni pozivaju funkciju `Morfoloske_Operacije`. Ta funkcija je prikazana u nastavku:

```
void Morfoloske_Operacije(int, void*) {
    int operation = morph_operator + 2;

    cv::Mat element = cv::getStructuringElement(morph_elem, cv::Size(2 *
        morph_size + 1, 2 * morph_size + 1), cv::Point(morph_size,
        morph_size));

    cv::morphologyEx(binarna, morfoloska_obrada, operation, element);

    cv::imshow(window_name, morfoloska_obrada);
}
```

Primijetit ćete da svaka od korisički napravljenih funkcija koristi prethodno spomenute globalne varijable. Razlog zašto je tako implementirano (inače nije preporučeno koristiti globalne varijable uvijek, ali ovdje imamo dobar razlog za to) je što bi funkcije bile previše glomazne kada bismo im prenosili sve potrebne parametre po referenci (radi uštede memorije i pamćenja promijene varijabli), tako da globalne varijable svakako čuvaju svaku promjenu koja se na njima desi u toku njihovog postojanja.

Prva linija funkcije `Morfoloske_Operacije` inicijalizira varijablu `int operation` preko globalne varijable `morph_operator` i dodaje joj 2. Nakon toga, pravimo novu instancu klase `cv::Mat` preko funkcije `cv::getStructuringElement` koja prima sljedeće parametre: prvi parametar je tipa `int` i predstavlja oblik elementa koji može biti jedan iz klase `cv::MorphShapes`; drugi parametar je tipa `cv::Size` i predstavlja veličinu strukturnog elementa; dok je treći parametar tipa `cv::Point` i predstavlja referentnu tačku unutar elementa.

Nakon toga pozivamo funkciju `cv::morphologyEx` koja prima sljedeće parametre: prvi parametar je tipa `InputArray` i predstavlja izvornu binarnu sliku; drugi parametar je tipa `OutputArray` i predstavlja sliku u koju se spašava rezultat napredne morfološke transformacije; treći parametar je tipa `int` i predstavlja tip morfološke operacije (detalje možete pronaći na linku); četvrti parametar je tipa `InputArray` i predstavlja strukturni element. Posljednja četiri parametra su preklopljena i u njihove detalje nećemo ulaziti.

Funkcija `Morfoloske_Operacije` je tu da uklonimo šum sa prosljeđene slike!

Iz *trackbar*-a iz `main` funkcije možemo vidjeti da mi zapravo pomjeranjem *trackbar*-a mijenjamo varijable `morph_operator`, `morph_elem` i `morph_size`. Pošto su poslani po referenci u funkciju `cv::getTrackbarPos` to znači da konstantno vršimo ažuriranje ova tri parametra (koji su globalne varijable).

Sljedeća stvar koju pozivamo u `main` funkciji je:

```
    Morfoloske_Operacije(0, 0);
    cv::waitKey(0);
```

čime pozivamo funkciju `Morfoloske_Operacije` i držimo prozor sa navedenim *trackbar*-ovima otvorenim, dokle god korisnik ne pritisne tipku na tastaturi.

Dalje ćemo objasniti funkciju za eroziju i šta smo u `main`-u napisali. `main` sada glasi:

```
    cv::namedWindow("Erozija - prikaz", cv::WINDOW_AUTOSIZE);

    cv::createTrackbar("Element:\n 0: Rect \n 1: Cross \n 2: Ellipse",
        "Erozija - prikaz", &erosion_elem, max_elem, Erozija);
    cv::createTrackbar("Kernel size:\n 2n +1", "Erozija - prikaz",
        &erosion_size, max_kernel_size, Erozija);

    Erozija(0, 0);
    cv::waitKey(0);
```

Slično kao za prethodno *user* definiranu funkciju, sada imamo korisničku funkciju `Erozija`:

```
void Erozija(int, void*) {
    int erosion_type = 0;
    if(erosion_elem == 0) erosion_type = cv::MORPH_RECT;
    else if(erosion_elem == 1) erosion_type = cv::MORPH_CROSS;
    else if(erosion_elem == 2) erosion_type = cv::MORPH_ELLIPSE;

    cv::Mat element = cv::getStructuringElement(erosion_type, cv::Size(2 *
        erosion_size + 1, 2 * erosion_size + 1), cv::Point(erosion_size,
        erosion_size));

    cv::erode(morfoloska_obrada, erozija, element);
    cv::imshow("Erozija - prikaz", erozija);
}
```

gdje na osnovu izabrane vrijednosti u *trackbar*-u biramo vrijednost za tip erozije. Opet koristimo na sličan način funkciju `cv::getStructuringElement` i onda pozivamo funkciju `cv::erode` koja prima sljedeće parametre: prvi parametar je tipa `InputArray` i predstavlja sliku nad kojom radimo eroziju; drugi parametar je tipa `OutputArray` i predstavlja sliku koja je rezultat erozije; treći parametar je tipa `InputArray` i predstavlja strukturni elemenat koji se koristi za eroziju; ostala četiri parametra su preklopljena i nećemo ulaziti u njihove detalje u ovom tutorijalu.

Sada radimo dilataciju na sličan način kao eroziju, u `main` funkciji pišemo:

```
    cv::namedWindow("Dilatacija - prikaz", cv::WINDOW_AUTOSIZE);
    cv::moveWindow("Dilatacija - prikaz", morfoloska_obrada.cols, 0);
    cv::createTrackbar("Element:\n 0: Rect \n 1: Cross \n 2: Ellipse",
        "Dilatacija - prikaz", &dilation_elem, max_elem, Dilatacija);
    cv::createTrackbar("Kernel size:\n 2n +1", "Dilatacija - prikaz",
        &dilation_size, max_kernel_size, Dilatacija);

    Dilatacija(0, 0);
    cv::waitKey(0);
```

dok funkcija `Dilatacija` izgleda ovako:

```
void Dilatacija(int, void*) {
    int dilation_type = 0;
    if(dilation_elem == 0) dilation_type = cv::MORPH_RECT;
    else if(dilation_elem == 1) dilation_type = cv::MORPH_CROSS;
    else if(dilation_elem == 2) dilation_type = cv::MORPH_ELLIPSE;

    cv::Mat element = cv::getStructuringElement(dilation_type, cv::Size(2 *
        dilation_size + 1, 2 * dilation_size + 1), cv::Point(dilation_size,
        dilation_size));

    cv::dilate(erozija, dilatacija, element);
    cv::imshow("Dilatacija - prikaz", dilatacija);
}
```

Vidimo ovdje i funkciju `cv::dilate` koja radi na sličan način kao funkcija `cv::erode`, samo što vrši dilataciju, a ne eroziju.

Sljedeći dio koda je najkompleksniji. Potrebno je sada na rezultantnu sliku erozije i dilatacije primijeniti *Watershed* algoritam. Kod `main` funkcije nakon svega do sada što smo prikazali glasi:

```
    cv::Mat konektovane(erozija.size(), CV_32S);
    int broj_komponenti = cv::connectedComponents(erozija, konektovane, 8,
        CV_32S);

    int sirina = konektovane.rows, duzina = konektovane.cols;
    int pix;

    for(int i = 0; i < sirina; i++) {
```

```
8           for(int j = 0; j < duzina; j++) {
9               pix = konektovane.at<int>(i, j);
10
11              if(pix == 0) {
12                  pix = broj_komponenti;
13
14                  konektovane.at<int>(i, j) = pix;
15              }
16
17          }
18      }
19
20      for(int i = 0; i < sirina; i++) {
21          for(int j = 0; j < duzina; j++) {
22              auto pix1 = erozija.at<char>(i, j);
23              auto pix2 = dilatacija.at<char>(i, j);
24
25              if(pix1 != pix2) {
26                  konektovane.at<int>(i, j) = 0;
27              }
28          }
29      }
30      cv::watershed(originalna, konektovane);
31
32      // Generate random colors:
33      std::vector<cv::Vec3b> colors;
34      for(int i = 0; i < broj_komponenti; i++) {
35          int b = cv::theRNG().uniform(0, 256);
36          int g = cv::theRNG().uniform(0, 256);
37          int r = cv::theRNG().uniform(0, 256);
38
39          colors.push_back(cv::Vec3b((uchar)b, (uchar)g, (uchar)r));
40      }
41
42      cv::Mat dst = cv::Mat::zeros(konektovane.size(), CV_8UC3);
43
44      for(int i = 0; i < konektovane.rows; i++) {
45          for(int j = 0; j < konektovane.cols; j++) {
46              int index = konektovane.at<int>(i, j);
47              if(index > 0 && index <= static_cast<int>(broj_komponenti)) {
48                  dst.at<cv::Vec3b>(i, j) = colors[index - 1];
49              }
50          }
51      }
```

Nakon što se uradi poziv funkcije `cv::connectedComponents` nad erodiranom slikom koja broji koliko novčića je na slici i tako izbrojane novčiće smješta u `cv::Mat konektovane` - najteži dio zapravo predstavlja kako da kombinacijom onoga što smo dobili kao rezultat i erozije i dilatacije kažemo zapravo programu za šta nismo sigurni da je pozadina ili novčić. To je urađeno u kodu u linijama 1 do 29.

Kada se tako dobije slika `konektovane` onda se `cv::watershed` primijeni nad njom i nad originalnom slikom. Onda se napravi vektor random boja za bojanje različitih novčića na slici i uz pomoć tog vektora i slike koja je rezultat `cv::watershed` funkcije možemo prikazati kako je program izdvojio različite novčiće sa slike. Ovo se ostavlja studentima da urade sami jednostavnom primjenom funkcije `cv::imshow` ili `cv::imwrite` kako bi vidjeli rezultate.

Watershed segmentacija radi na principu markera. Svakom pikselu će se dodijeliti marker koji odgovara ili pozadini ili nekom od objekata na slici. Algoritam *Watershed* segmentacije zahtjeva da mu se pripremi slika tako što se markerima označe neki pikseli koji sigurno pripa-

daju svakom od objekata (minimalno 1 piksel po objektu), neki pikseli koji sigurno pripadaju pozadini (minimalno 1 piksel za svaki nepovezani dio pozadine) i da preostali pikseli budu označeni sa markerom 0 koji označava da nije unaprijed poznato čemu pripadaju. *Watershed* segmentacija će promijeniti markere na ovim pikselima i dodijeliti ih nekom od objekata ili pozadini.

Watershed segmentacija se zasniva na inkrementalnom dodjeljivanju markera nepoznatim pikselima vrijednost markera nekog od susjednih piksela. Iz ovog razloga je preprocesiranje slike dodjeljivanjem nekih tačnih markera od ključne važnosti i zbog toga smo imali tri funkcije koje su pripremile datu sliku za *Watershed* segmentaciju. Napominjemo da studenti ne zaborave sve module i biblioteke koje se trebaju uključiti na početku programa kako bi im radio primjer.

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.